

A Brief Report on StoryboardAgent for Four-Panel Comic Story Exp

Abstract

This report summarizes the design of StoryboardAgent, a lightweight planning component for a CLI-based four-panel comic generator. The goal is to transform short user themes into exactly four concrete, visually distinct panel beats before prompt construction and image generation. The design stays deterministic by default, preserves the existing pipeline contract, and improves story clarity for common prompt types such as day-in-the-life, seasonal progression, journey, and problem-solving themes.

Introduction

The comic agent project converts a short theme into a four-panel comic package, including panel prompts, generated images, a composed final page, and metadata. Earlier story planning could produce panels that were structurally correct but visually repetitive. This weakens narrative progression and makes downstream image prompts less informative. StoryboardAgent addresses this issue by inserting a focused planning layer between the user request and prompt generation. Instead of repeating the same theme across four panels, the agent identifies a simple story structure, resolves the main subject, and expands the request into four single-scene panel descriptions. The main design goals are simplicity, determinism, and compatibility with the current CLI workflow.

Related Work

Within this repository, the most relevant prior work includes three existing subsystems. First, the baseline story planner already produced a StoryStructure consumed by downstream prompt building. Second, the LM-assisted planner added a semantic interpretation path for richer understanding of prompts. Third, the panel validation pipeline enforced the single-panel image contract after generation. StoryboardAgent does not replace these components. It becomes the planning layer that creates stronger panel-level story beats while keeping the old entrypoint and output schema intact. Compared with a fully LM-driven planner, the default rule-based design trades flexibility for controllability, reproducibility, and testability.

Method

The method follows a narrow and explicit pipeline. Given a ComicRequest, the agent first normalizes the theme and resolves the subject. If the user provides an explicit character override, that value has highest priority. Otherwise, the agent performs keyword-based subject extraction and prefers specific subjects such as stray orange cat over generic nouns such as cat. Next, the agent derives lightweight semantics from the theme, including activity, primary object, setting hints, and domain cues. These cues drive deterministic intent detection. The current implementation supports at least five core structures: day-in-the-life, seasonal progression, journey or adventure, problem solving, and fallback. The codebase also keeps a competition structure for sports-like prompts. Once the

structure is selected, the agent expands the request into four fixed beats. Each beat contains a caption, action, emotion, and camera framing, and is rendered into a full PanelSpec with concrete scene and visual descriptions. The method explicitly avoids page-level layout terms such as grid, comic page, or four-panel layout, because each panel must remain a single image scene for downstream generation. The system also preserves compatibility with LM-assisted planning. In that mode, an external planner may provide one structured semantic or storyboard result, but the local pipeline still validates the result and keeps the same downstream interfaces. If LM-assisted planning fails, the system falls back to the deterministic rule-based path.

Experimental Results

Evaluation in this project is regression-oriented rather than benchmark-oriented. The repository includes unit tests for subject extraction, semantic parsing, intent detection, and story expansion, as well as integration tests for full mock-provider pipeline execution. The available tests show that the planner produces the expected four-stage progressions for representative prompts. For example, the school-day prompt maps to Morning, On the Way, At School, and Going Home; the seasonal cat prompt maps to Spring, Summer, Autumn, and Winter; the lantern prompt maps to Departure, Encounter, Turning Point, and Resolution; and the city-light repair prompt maps to Problem, Search, Solution, and Payoff. The tests also check that panel scene descriptions stay distinct and avoid prohibited layout language. Integration coverage further indicates that the planner remains compatible with the existing CLI workflow. In mock runs, the pipeline still produces four panel images, one final comic image, and a metadata file with planner traces. Additional tests verify that unsafe themes are rejected early, LM-assisted results can be consumed when valid, and retry traces are recorded when panel validation rejects an image attempt. These results suggest that the main benefit of StoryboardAgent is improved structural reliability before prompt generation, which should reduce repetitive prompts and strengthen character consistency in downstream image synthesis.

Conclusion

StoryboardAgent is a small but high-leverage addition to the comic generation pipeline. By converting short prompts into concrete four-panel story beats, it improves narrative clarity without changing the CLI interface, provider abstractions, or export format. Its deterministic default path is easy to test, easy to reason about, and appropriate for an MVP-style local tool. Future work can improve semantic coverage and LM-assisted robustness, but the current design already establishes a strong planning boundary: one request, one subject, one structure, and four clear visual moments.